LABORATORIO

Protocolo MQTT

preparado por Prof. Antonio Russoniello

1. Introducción

En este laboratorio configuraremos un servidor para el intercambio de mensajes M2M. Haremos uso del protocolo MQTT, específicamente de la implementación conocida como Mosquitto.

2. Instrucciones

Utilice para el servidor y el cliente una placa **Raspberry Pi** disponible en el laboratorio o en su defecto una maquina virtual. En los pasos que se indicarán a continuación se asume que el sistema operativo del servidor es Linux 64 bits.

2.1. Sin Autenticación

Pasos en la máquina del Servidor
Instalamos desde el servidor ejecutando el comando:

```
sudo apt install mosquitto -y
```

procedemos a configurar puertos, protocolo y autenticación

```
sudo nano /etc/mosquitto/mosquitto.conf
```

agregar las siguientes líneas al archivo

```
listener 1883  # puerto por defecto
# habilitar websockets, opcional
listener 9001
protocol websockets
# permitir usuarios anonimos
allow_anonymous true
```

procedemos a iniciar el servidor mosquitto:

```
sudo systemctl start mosquitto
sudo systemctl enable mosquitto
```

verificamos que el servicio se encuentre habilitado

```
sudo systemctl status mosquitto
```

Pasos en la máquina Cliente
A continuación procedemos a instalar el cliente mosquitto en otro PC (o máquina host de la máquina virtual, si fuera el caso).

```
sudo apt install mosquitto-clients -y
```

```
russo@russo-VirtualBox:~$ sudo systemctl status mosquitto.service
● mosquitto.service - Mosquitto MQTT Broker
     Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
     Active: active (running) since Sun 2025-01-12 14:15:07 -04; 4s ago
       Docs: man:mosquitto.conf(5)
             man:mosquitto(8)
    Process: 3373 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 3375 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)
    Process: 3378 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)
    Process: 3380 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)
   Main PID: 3382 (mosquitto)
      Tasks: 1 (limit: 2269)
     Memory: 2.1M (peak: 2.4M)
        CPU: 31ms
     CGroup: /system.slice/mosquitto.service
             └─3382 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

Jan 12 14:15:07 russo-VirtualBox systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
Jan 12 14:15:07 russo-VirtualBox systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.
```

Figura 1: Estatus del Servicio mosquitto en el Servidor

Poniendo a prueba la instalación:

Asegúrese de habilitar en el firewall, tanto del servidor como del cliente, los puertos del servicio.

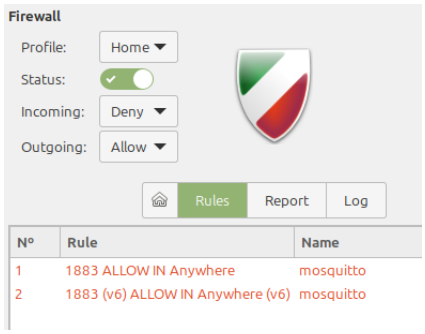


Figura 2: Reglas del Firewall

Si está usando una máquina virtual, debe seleccionar en “Network Setting” la opción “Bridged Adapter”

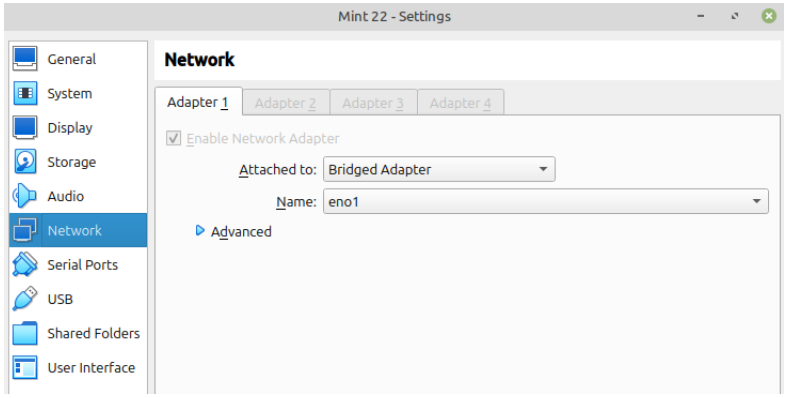


Figura 3: Configuración de Red para la Máquina Virtual en VirtualBox

desde el lado del cliente ejecute los siguientes comandos (desde la terminal)

```
mosquitto_sub -h IP_SERVIDOR -t test
```

abra una nueva terminal desde el mismo cliente y ejecute el comando
`mosquitto_pub -h IP_SERVIDOR -t test -m "mensaje de prueba"`

Con la primera instrucción definimos una suscripción a la etiqueta "test" y con la segunda instrucción publicamos el mensaje "mensaje de prueba" y lo asociamos a la etiqueta "test". Como estamos suscritos a la etiqueta "test" todo lo que se envie a ella nos será mostrado en la terminal. Lo que está ocurriendo es que el mensaje publicado llega al servidor mosquitto y es este el que se encarga de reenviar los mensajes a los clientes en función de las sucripciones de estos a etiquetas específicas.

Repita la experiencia enviando mensajes desde otros clientes y utilizando distintas etiquetas. Utilice como clientes los PCs disponibles en el laboratorio. Reciba en cada cliente mensajes del resto de los clientes de la red, por ejemplo cree las etiquetas "cliente1" para mensajes enviados desde la máquina CLIENTE1, "cliente2" para mensajes enviados desde la máquina CLIENTE2 y "cliente3" para mensajes enviados desde la máquina CLIENTE3 y haga que cada máquina reciba mensajes de las otras estaciones clientes.

2.2. Autenticación por Usuario/Clave

Pasos en la máquina del Servidor
Desde el servidor creamos el archivo con el usuario y la clave:
`sudo mosquitto_passwd -c /etc/mosquitto/passwd ALIAS_USUARIO`

procedemos a configurar archivo mosquitto.conf
`sudo nano /etc/mosquitto/mosquitto`
agregamos las siguientes opciones

```
allow_anonymous false
password_file  /etc/mosquitto/passwd
```

modificamos los permisos de acceso al archivo "passwd" creado previamente
`sudo chmod 755 /etc/mosquitto/passwd`
reiniciamos el servicio
`sudo systemctl restart mosquitto`

Poniendo a prueba la instalación:
desde el cliente ejecute los siguientes comandos (desde la terminal)
`mosquitto_sub -h IP_SERVIDOR -t test -u "usuario" -P "password"`

abra una nueva terminal desde el mismo cliente y ejecute el comando
`mosquitto_pub -h IP_SERVIDOR -t test -m "mensaje prueba" -u "usuario" -P "password"`

2.3. Publicación y Suscripción por Código

A continuación vamos a realizar la publicación y la suscripción de mensajes mosquitto a través de código en Python. Se muestra un ejemplo con las funciones básicas usando la librería paho.

```
import paho.mqtt.client as mqtt
import random

broker, port, topic, client_id, username, password = \
   '190.169.74.69', 1883, 'etiqueta', \
   f'publish-{random.randint(0,100)}', 'pedroperez', '12345'

# The callback for when the client receives a CONNACK response
# from the server.
def on_connect(client, userdata, flags, reason_code, properties):
   print(f'Connected with result code {reason_code}')
   # Subscribing in on_connect() means that if we lose the
   # connection and reconnect then subscriptions will be renewed.
   client.subscribe('TOPIC')

# The callback for when a PUBLISH message is received
# from the server.
def on_message(client, userdata, msg):
   print(msg.topic+' '+str(msg.payload))

# MAIN #
mqttc = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
mqttc.password=password
mqttc.username=username
mqttc.on_connect = on_connect
mqttc.on_message = on_message
mqttc.connect(broker, port)

# Blocking call that processes network traffic, dispatches
# callbacks and handles reconnecting.
# Other loop*() functions are available that give a
# threaded interface and a manual interface.
mqttc.loop_forever()
```

3. Actividades

3.1. Conecte un sensor de ultrasonido a la placa raspberry pi, muestre el circuito en Fritzing y agregue una foto de su prototipo. (**5 puntos**)

3.2. Modifique el código mostrado para que cada 10 segundos se realize una lectura del sensor de ultrasonido y se muestre la salida en la terminal. (**5 puntos**)

```
import RPi.GPIO as GPIO
import time

# Configuracion de pines (Modo BCM)
TRIG = 23 # Conectado al pin fisico 16
ECHO = 24 # Conectado al pin fisico 18 (con divisor de voltaje)
```

```python
# Configurar la numeracion de los pines de la placa
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

def obtener_distancia():
    # Asegurar que el pin TRIG comience apagado
    GPIO.output(TRIG, False)
    time.sleep(0.5)

    # Enviar un pulso ultrasonico de 10 microsegundos
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    # Registrar el inicio del pulso de retorno (ECHO se pone en ALTO)
    while GPIO.input(ECHO) == 0:
        inicio_pulso = time.time()

    # Registrar el fin del pulso de retorno (ECHO vuelve a BAJO)
    while GPIO.input(ECHO) == 1:
        fin_pulso = time.time()

    # Calcular la diferencia de tiempo
    duracion = fin_pulso - inicio_pulso

    # Distancia = (Tiempo x Velocidad del sonido (34300 cm/s)) / 2 (ida y vuelta)
    distancia = (duracion * 34300) / 2
    return round(distancia, 2)

try:
    print('Midiendo distancia... Presiona Ctrl+C para detener.')
    while True:
        dist = obtener_distancia()
        print(f'Distancia: {dist} cm')
        time.sleep(1) # Esperar 1 segundo entre lecturas

except KeyboardInterrupt:
    print('\nLectura interrumpida.')
    GPIO.cleanup() # Limpiar la configuracion de los pines GPIO
```

3.3. Modifique el código para que las lecturas del sensor sean publicadas al broker MQTT con el tópico /GRUPO/SENSOR . Muestre la salida de la terminal del broker. (**5 puntos**)

3.4. Modifique el código anterior para publicar el tópico al broker de otro de los grupos de trabajo. (**5 puntos**)

§

Total de puntos: 20